

Safe Wealth Advisory & Governed Portfolio Rebalancer

Complete Production Repository for Lyzr Agent API & Safe AI Challenge

Track: **Lyzr Solo Quest**

Domain: **WealthTech / Wealth Mgmt**

Compliance: **FINRA / SEC Fiduciary**

1. Repository Directory Structure

```
safe-wealth-advisor/  
├─ requirements.txt # Core dependencies & SDKs  
├─ config.py # Environmental settings & safety limits  
├─ models.py # Pydantic schemas (IPS, Portfolio, Trades)  
├─ rebalancer.py # Deterministic Portfolio Math Engine  
├─ safe_guard.py # Lyzr Safe AI Suitability Filter  
├─ agents.py # Lyzr Agent API Orchestration  
├─ aims_logger.py # SEC Audit Trail via Lyzr AIMS  
├─ main.py # FastAPI Service Entrypoint  
└─ tests/test_rebalancer.py # Suite of Pytest Verification Checks
```

2. Production Source Code

requirements.txt

```
lyzr-agent-api>=0.1.25  
lyzr-safe-ai>=0.0.8  
fastapi>=0.104.0  
uvicorn>=0.23.2  
pydantic>=2.4.2  
numpy>=1.26.0  
scipy>=1.11.3  
pytest>=7.4.2  
python-dotenv>=1.0.0  
requests>=2.31.0
```

config.py

```
import os  
from pydantic_settings import BaseSettings  
  
class Settings(BaseSettings):  
    APP_NAME: str = "Safe Wealth Advisory & Governed Rebalancer"  
    ENVIRONMENT: str = os.getenv("ENVIRONMENT", "production")  
  
    # Lyzr API Keys  
    LYZR_API_KEY: str = os.getenv("LYZR_API_KEY", "lyzr_live_secret_key")  
    LYZR_AIMS_ENDPOINT: str = os.getenv("LYZR_AIMS_ENDPOINT", "https://aims.lyzr.ai/v1/audit")  
  
    # Fiduciary Boundaries  
    MIN_CASH_BUFFER_PCT: float = 0.05 # Mandatory 5% liquidity reserve  
    MAX_EQUITY_CONSERVATIVE: float = 0.30 # Conservative IPS equity ceiling  
    PROHIBITED_ASSETS: list = ["MEME_COIN", "UNCOVERED_OPTION", "HIGH_LEVERAGE ETF"]
```

```
class Config:
    env_file = ".env"

settings = Settings()
```

models.py

```
from pydantic import BaseModel, Field
from typing import List, Dict, Optional

class Holding(BaseModel):
    symbol: str
    asset_class: str # Equity, Fixed Income, Commodity, Cash
    shares: float
    current_price: float
    cost_basis: float

class IPSProfile(BaseModel):
    client_id: str
    risk_score: int = Field(..., ge=1, le=10) # 1 (Conservative) to 10 (Aggressive)
    time_horizon_years: int
    liquidity_needs_usd: float
    tax_bracket_pct: float
    prohibited_sectors: List[str] = []

class Portfolio(BaseModel):
    client_id: str
    holdings: List[Holding]

class TradeOrder(BaseModel):
    symbol: str
    action: str # BUY, SELL, HOLD
    shares: float
    estimated_price: float
    tax_impact_usd: float
    reasoning: str

class RebalanceProposal(BaseModel):
    client_id: str
    target_allocations: Dict[str, float]
    orders: List[TradeOrder]
    compliance_approved: bool
    audit_id: str
```

rebalancer.py

```
import numpy as np
from typing import Dict, List
from models import Portfolio, IPSProfile, TradeOrder

class DeterministicRebalancer:
    # Deterministic portfolio optimization engine. Ensures math precision
    # without non-deterministic LLM hallucinations in financial calculations.

    @staticmethod
    def compute_target_allocation(ips: IPSProfile) -> Dict[str, float]:
        # Rule-based asset allocation matrix based on Risk Score
        if ips.risk_score <= 3:
            return {"Equity": 0.20, "Fixed Income": 0.65, "Commodity": 0.05, "Cash": 0.10}
        elif ips.risk_score <= 7:
            return {"Equity": 0.60, "Fixed Income": 0.30, "Commodity": 0.05, "Cash": 0.05}
```

```

        else:
            return {"Equity": 0.80, "Fixed Income": 0.10, "Commodity": 0.05, "Cash": 0.05}

@classmethod
def generate_orders(cls, portfolio: Portfolio, ips: IPSProfile) -> List[TradeOrder]:
    total_val = sum(h.shares * h.current_price for h in portfolio.holdings)
    targets = cls.compute_target_allocation(ips)
    orders = []

    for h in portfolio.holdings:
        current_val = h.shares * h.current_price
        target_val = total_val * targets.get(h.asset_class, 0.0)
        diff_val = target_val - current_val

        if abs(diff_val) > 500.0: # Minimum trade execution threshold
            trade_shares = diff_val / h.current_price
            action = "BUY" if trade_shares > 0 else "SELL"

            # Compute estimated tax implications for sells
            tax_impact = 0.0
            if action == "SELL":
                gain_per_share = max(0.0, h.current_price - h.cost_basis)
                tax_impact = abs(trade_shares) * gain_per_share * (ips.tax_bracket_pct / 100.0)

            orders.append(TradeOrder(
                symbol=h.symbol,
                action=action,
                shares=round(abs(trade_shares), 2),
                estimated_price=h.current_price,
                tax_impact_usd=round(tax_impact, 2),
                reasoning=f"Rebalancing {h.asset_class} to target weight
{targets.get(h.asset_class)*100}%"
            ))

    return orders

```

safe_guard.py

```

from config import settings
from models import IPSProfile, TradeOrder
from typing import List, Tuple

class LyzrSafeAIGuard:
    # FINRA Suitability & SEC Fiduciary Safety Gate.
    # Filters proposals against strict policy limits before advisor display.

    @staticmethod
    def validate_suitability(ips: IPSProfile, orders: List[TradeOrder]) -> Tuple[bool, List[str]]:
        violations = []

        for order in orders:
            # Block 1: Prohibited Asset Screening
            if order.symbol in settings.PROHIBITED_ASSETS:
                violations.append(f"CRITICAL: Trade order for {order.symbol} violates Safe AI Prohibited List.")

            # Block 2: Prohibited Sector Screening
            if any(sector in order.reasoning for sector in ips.prohibited_sectors):
                violations.append(f"SUITABILITY VIOLATION: {order.symbol} matches IPS sector restrictions.")

            # Block 3: Conservative Risk Protection
            if ips.risk_score <= 3 and order.action == "BUY":

```

```

        if order.tax_impact_usd > 5000.0:
            violations.append(f"TAX GUARD: Tax drag of ${order.tax_impact_usd} exceeds
Conservative limit.")

    is_approved = len(violations) == 0
    return is_approved, violations

```

agents.py

```

from lyzr_agent_api import AgentAPI
from config import settings

class WealthAdvisorAgents:
    def __init__(self):
        self.client = AgentAPI(api_key=settings.LYZR_API_KEY)

    def get_fiduciary_profiler_agent(self):
        return self.client.create_agent(
            name="Fiduciary Profiler Agent",
            system_prompt=(
                "You are an expert Wealth Management Fiduciary. Analyze the client's IPS, "
                "horizon, and tax status to deliver suitability briefings adhering to FINRA Rule
2111."
            )
        )

    def generate_advisor_brief(self, agent_id: str, client_id: str, orders_summary: str) -> str:
        response = self.client.run_agent(
            agent_id=agent_id,
            message=f"Generate trade proposal briefing for Client {client_id}. Orders:
{orders_summary}"
        )
        return response.get("content", "Briefing generation completed.")

```

aims_logger.py

```

import requests
import uuid
import json
from config import settings

class AIMSLogger:
    # Lyzr AIMS (Agent Intelligence Management System) Audit Trail
    # Maintains immutable compliance logs for SEC examinations.

    @staticmethod
    def log_rebalance_event(client_id: str, orders: list, suitability_passed: bool, notes: list) -> str:
        audit_id = f"SEC-AUDIT-{uuid.uuid4().hex[:8].upper()}"
        payload = {
            "audit_id": audit_id,
            "client_id": client_id,
            "suitability_passed": suitability_passed,
            "orders_count": len(orders),
            "compliance_notes": notes,
            "timestamp": "2026-09-11T21:28:36Z"
        }

        try:
            # Post audit record to Lyzr AIMS persistence ledger
            requests.post(settings.LYZR_AIMS_ENDPOINT, json=payload, timeout=3.0)

```

```

    except Exception:
        # Fallback local audit trace
        with open("sec_compliance_audit.log", "a") as f:
            f.write(json.dumps(payload) + "
")

    return audit_id

```

main.py

```

from fastapi import FastAPI, HTTPException
from models import IPSProfile, Portfolio, RebalanceProposal
from rebalancer import DeterministicRebalancer
from safe_guard import LyzrSafeAIGuard
from aims_logger import AIMSLogger
from pydantic import BaseModel

app = FastAPI(title="Safe Wealth Advisory API", version="1.0.0")

class RebalanceRequest(BaseModel):
    ips: IPSProfile
    portfolio: Portfolio

@app.post("/api/v1/rebalance", response_model=RebalanceProposal)
def execute_rebalance(req: RebalanceRequest):
    # 1. Deterministic Optimization
    orders = DeterministicRebalancer.generate_orders(req.portfolio, req.ips)

    # 2. Lyzr Safe AI Suitability Gate
    passed, violations = LyzrSafeAIGuard.validate_suitability(req.ips, orders)

    # 3. SEC Audit Logging via Lyzr AIMS
    audit_id = AIMSLogger.log_rebalance_event(
        client_id=req.ips.client_id,
        orders=[o.model_dump() for o in orders],
        suitability_passed=passed,
        notes=violations
    )

    if not passed:
        raise HTTPException(status_code=422, detail={"suitability_violations": violations})

    return RebalanceProposal(
        client_id=req.ips.client_id,
        target_allocations=DeterministicRebalancer.compute_target_allocation(req.ips),
        orders=orders,
        compliance_approved=passed,
        audit_id=audit_id
    )

```

tests/test_rebalancer.py

```

import pytest
from models import IPSProfile, Portfolio, Holding
from rebalancer import DeterministicRebalancer
from safe_guard import LyzrSafeAIGuard

def test_deterministic_rebalance_math():
    ips = IPSProfile(
        client_id="C-101", risk_score=2, time_horizon_years=10,
        liquidity_needs_usd=10000.0, tax_bracket_pct=24.0
    )

```

```

    )
    portfolio = Portfolio(
        client_id="C-101",
        holdings=[
            Holding(symbol="VTI", asset_class="Equity", shares=100, current_price=200.0, cost_basis=15
0.0),
            Holding(symbol="BND", asset_class="Fixed Income", shares=50, current_price=80.0,
cost_basis=80.0)
        ]
    )
    orders = DeterministicRebalancer.generate_orders(portfolio, ips)
    assert len(orders) > 0
    assert orders[0].action == "SELL" # Rebalancing away high equity ratio for Conservative client

def test_safe_ai_prohibited_asset():
    ips = IPSProfile(
        client_id="C-102", risk_score=5, time_horizon_years=5,
        liquidity_needs_usd=5000.0, tax_bracket_pct=15.0
    )
    from models import TradeOrder
    invalid_orders = [
        TradeOrder(symbol="MEME_COIN", action="BUY", shares=100, estimated_price=10.0,
tax_impact_usd=0.0, reasoning="Speculative")
    ]
    passed, violations = LyzrSafeAIGuard.validate_suitability(ips, invalid_orders)
    assert passed == False
    assert "CRITICAL" in violations[0]

```